

AWS GenAI 課程 · 第 3 單元

Docker 多系統兼容與 GitHub 多分支管理

盧建霖 • 2026-09-17

「在我電腦上可以跑。」
這個單元要讓這句話沒有意義。

Agenda

- 01 容器與映像
- 02 映像怎麼做、容器怎麼接
- 03 分支策略與合併衝突
- 04 AI 環境與稽核實作

TODAY'S GOAL

寫得出一份交得出去的
映像，管得住一條合得回
去的分支。

前兩段是 Docker，第三段是 Git，最後一段把兩邊接起來。

先定義名詞，後面才好講

這六個詞今天會反覆出現，第一次出現就先給名字

核心 **kernel**

作業系統最底層，直接管硬體與行程的那一塊。

虛擬機 **VM**

用軟體模擬一整台電腦，裡面裝自己的作業系統。

容器 **container**

只帶自己的檔案系統與程式庫，借用主機核心跑的隔離環境。

映像 **image**

容器的唯讀範本；容器是從映像啟動出來的執行個體。

層 **layer**

Dockerfile 一條指令產生的一組檔案變化，疊起來就是映像。

快取 **cache**

Docker 記住上次每一層的結果，輸入沒變化就直接拿舊的。

容器與虛擬機差在哪

- 一、 兩張疊層圖：差別只在核心那一層
- 二、 快、小、隔離較弱，三個特性一個原因
- 三、 雲端上兩者是疊著用的，不是二選一

差別只有一層：核心共不共用

左邊虛擬機，右邊容器；紅色那層是每台各自的作業系統核心

虛擬機：每台各自帶一個作業系統核心



容器：共用主機核心，只隔離看得到的東西



紅色那層就是差別：VM 每台多背一個核心，所以慢、大、但隔得乾淨；容器共用核心，所以快、小、但隔離較弱

- 1 VM 每台各帶一個核心，所以要開機、要幾個 GB。
- 2 容器少了那一層，啟動只是開一個被隔離的行程。
- 3 代價：容器裡的程式必須跟主機的架構相容。

三個特性，一個原因

特性	為什麼	代價
啟動快	不用開機，只是啟動一個被隔離的行程	主機核心掛了，所有容器一起掛
映像小	不用包作業系統核心，只包程式庫與程式	映像跟主機架構綁在一起
隔離較弱	大家共用同一個核心，隔離靠核心的功能	需要強隔離時還是用 VM

一句話帶走：容器不是輕的 VM，是借用核心的隔離行程。

容器比較新，所以 **VM** 過時了？

✗ 大家通常會想

容器比 **VM** 先進，所以 **VM** 被淘汰了，兩者是二選一。

這個想法會讓你在雲端上看不懂自己的容器跑在哪。

✓ 其實

雲端上你的容器多半跑在一台 **VM** 裡面，兩者是疊著用的。

VM 負責跟別的客戶隔開，容器負責讓你的程式到哪都一樣。

QUIZ 01

為什麼 **linux/amd64** 的容器不能直接在 **arm64** 的機器上跑？

A

映像太大

B

容器共用主機核心，程式要跟主機架構相容

C

Docker 不支援 arm

D

需要先開一台 VM

想 30 秒，等一下翻下一頁對答案。

QUIZ 01 · ANSWER

為什麼 **linux/amd64** 的容器不能直接在 **arm64** 的機器上跑？

A 映像太大

B 容器共用主機核心，程式要跟主機架構相容

C Docker 不支援 arm

D 需要先開一台 VM

B。這是 Docker 文件的原句：容器共用主機核心，所以裡面的程式必須跟主機架構相容。第四段會講 **buildx** 建多平台映像。

BUILD & RUN

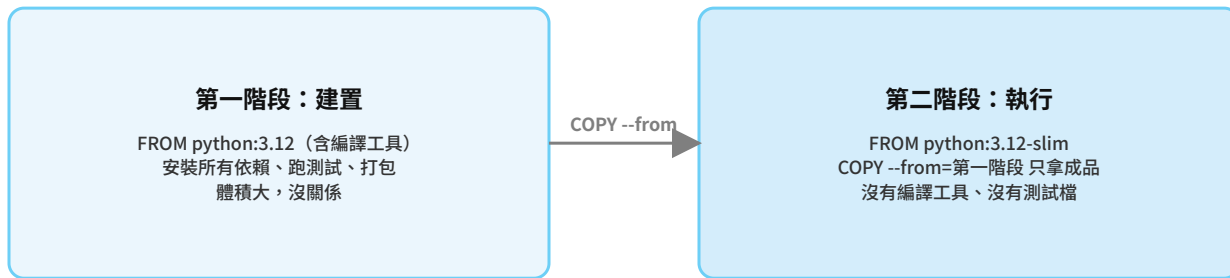
映像怎麼做、容器怎麼接

- 一、 **Dockerfile** 的四件事：多階段、順序、非 **root**、最小基底
- 二、 **Layer** 與快取：只改一行，為什麼全部重裝
- 三、 **Volume** 與 **Port**：連不到、資料不見，先查哪條線

建的地方跟跑的地方分開

一個 Dockerfile 裡寫多個 FROM，最後一個階段才是交出去的映像

一個 Dockerfile、多個 FROM，最後一個 FROM 才是交出去的映像



建置用的東西留在第一階段，最終映像只剩跑得起來需要的

- 1 第一階段帶編譯工具與測試，體積大沒關係。
- 2 第二階段只用 **COPY --from** 把成品搬過來。
- 3 沒被 **COPY** 過來的東西，就留在後面不上線。

一份兩階段、非 root 的 Dockerfile

```
Dockerfile

FROM python:3.12-slim AS build
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.12-slim
RUN useradd -m appuser
WORKDIR /app
COPY --from=build /install /usr/local
COPY app.py .
USER appuser
CMD ["python", "app.py"]
```

建置 `docker build -t myapp .` 執行 `docker run --rm myapp` 確認身分 `docker run --rm myapp whoami`

這樣寫會出事，改成右邊

四個最常見的寫法問題，對照著看

✗ 大家通常這樣寫

FROM python:latest

一個階段從頭裝到尾

沒寫 USER，程式以 root 跑

用完整版映像圖方便

✓ 其實應該

釘版本，例如 `python:3.12-slim`；標籤會被更新，三個月後可能指到不同版本

建置一個階段，執行一個階段，編譯工具不進最終映像

建一個使用者，最後 USER 切過去；隔離較弱，被攻破就是 root

`slim` 夠用就用 `slim;alpine` 為了小，連 `git`、`bash` 都不含

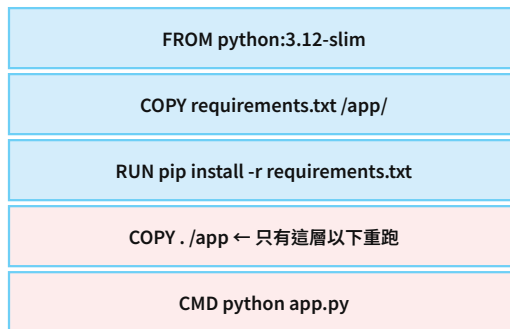
一層變了，後面全部重建

藍色是快取命中，紅色是重新執行；差別只在 COPY 原始碼放在哪

COPY 放太前面：改一行程式，全部重裝



先 COPY 依賴清單：改程式只重跑最後兩層



規則只有一條：某一層變了，它後面的每一層都要重建，就算內容一樣

藍色＝快取命中，紅色＝重新執行；把最常變的東西放到最下面

- 1 順序是拿「多久變一次」排的，不是拿邏輯順序排的。
- 2 依賴清單先於安裝，原始碼放最下面。
- 3 層可以在映像之間共用，所以第二個應用建得更快。

跟著做：量一次快取

四次建置，拿到四個時間；用 `time` 量或看 `build` 輸出的秒數

1

正確順序建一次，記時間 **A**

```
$ time docker build -t myapp .
```

2

改一行程式再建，記時間 **B**

```
$ echo "# touch" >> app.py && time docker build -t myapp .
```

`pip install` 那層應該顯示 `CACHED`，時間掉到幾秒

3

把 **COPY app.py** 搬到 **pip install** 之前，建一次當新基準（時間 **C**）

```
$ time docker build -t myapp-bad .
```

4

再改一行程式建一次，記時間 **D**，比較 **B** 與 **D**

```
$ echo "# touch" >> app.py && time docker build -t myapp-bad .
```

D 明顯慢於 **B**，因為順序錯了之後 `pip install` 被拖著重跑

QUIZ 02

第 3 層是 **COPY ./app**，第 4 層是 **RUN pip install**。
改了一行程式後重建，哪些層重跑？

A 只有第 3 層

B 第 3 層與之後全部

C 全部五層

D 都不重跑，pip 有快取

想 30 秒，等一下翻下一頁對答案。

QUIZ 02 · ANSWER

第 3 層是 **COPY ./app**，第 4 層是 **RUN pip install**。
改了一行程式後重建，哪些層重跑？

A 只有第 3 層

B 第 3 層與之後全部

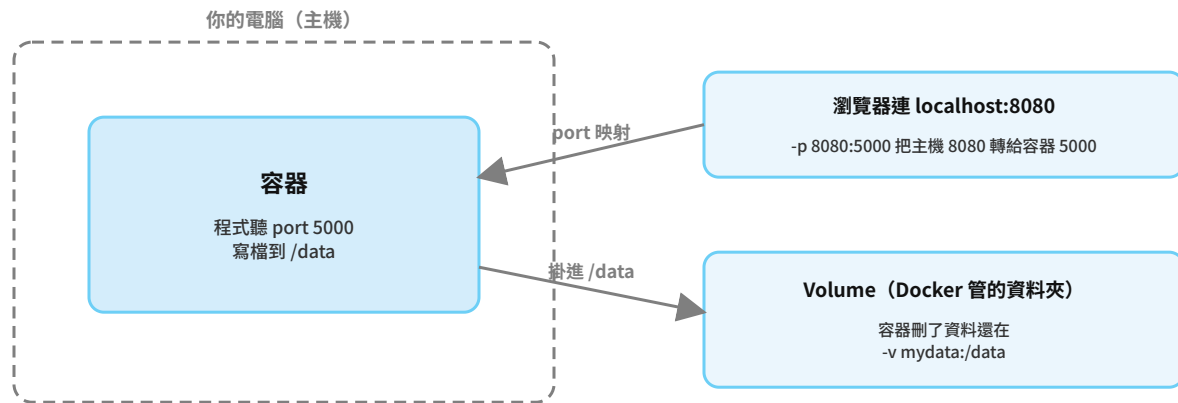
C 全部五層

D 都不重跑，pip 有快取

B。第 3 層的輸入變了，它與之後的層全部重建；pip 自己的快取跟 Docker 的層快取是兩件事。

容器跟外面只有兩條線

上面是 **port** 映射，把主機的 8080 轉給容器的 5000；下面是 **Volume**，掛進容器的 **/data**



容器裡的 port 與路徑是它自己的世界，要跟外面接上得靠映射與掛載；「在我電腦上可以跑」多半是這兩條線沒接對

- 1 -p 的順序是主機在前，預設對所有網路介面開放。
- 2 要留的資料一律放 **Volume**，官方說這是持久化首選。
- 3 開發時要即時改程式，用 **bind mount** 把原始碼掛進去。

跟著做：把兩條線接錯，再接對

程式是一支聽 5000 埠、把計數寫到 /data 的小服務

1

先在 **USER** 之前補目錄權限，不然非 **root** 寫不進 /data

```
RUN mkdir -p /data && chown appuser /data
```

2

實驗一：不加 **-p** 跑，瀏覽器連不到；加了就連得到

```
$ docker run --rm myapp  
$ docker run --rm -p 8080:5000 myapp
```

連 localhost:8080；程式要聽 0.0.0.0，聽 127.0.0.1 只有容器自己連得到

3

實驗二：不掛 **Volume**，刪掉容器重開，計數歸零

```
$ docker run -d -p 8080:5000 --name c1 myapp  
$ docker rm -f c1 && docker run -d -p 8080:5000 --name c1 myapp
```

4

加上 **-v** 再做一遍，刪掉重開後計數接著算

```
$ docker run -d -p 8080:5000 -v mydata:/data --name c1 myapp
```

排錯順序記住：先查埠、再查路徑，然後才是程式

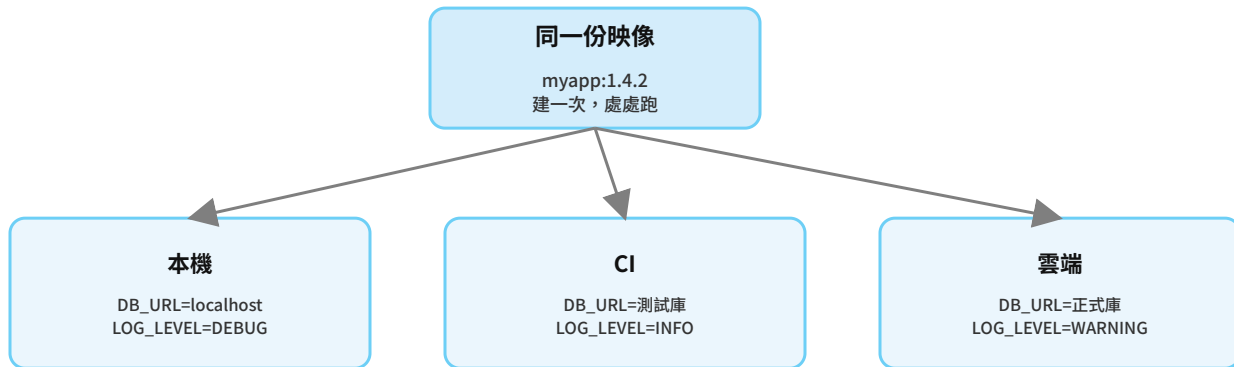
資料要留下來，有三種放法

方式	是什麼	什麼時候用
Volume	Docker 自己管理的資料夾，容器刪了它還在	資料庫檔案、上傳的檔案；官方的持久化首選
Bind mount	把主機上某個路徑直接掛進容器	開發時把原始碼掛進去即時改
tmpfs	只放在記憶體，容器停了就沒	暫存、機密，不想落地

一句話帶走：容器是**用完即丟**的，資料與入口都要接到外面。

映像不變，變的只有環境變數

一個帶版本號的映像，三個環境；三個框裡不同的只有環境變數的值



映像不變，變的只有環境變數與設定檔；環境差異收斂到這一層，就不會「本機可以、雲端不行」

進映像

程式碼、程式庫、執行環境；每個環境都一樣的東西。改了要重建。

留在外面

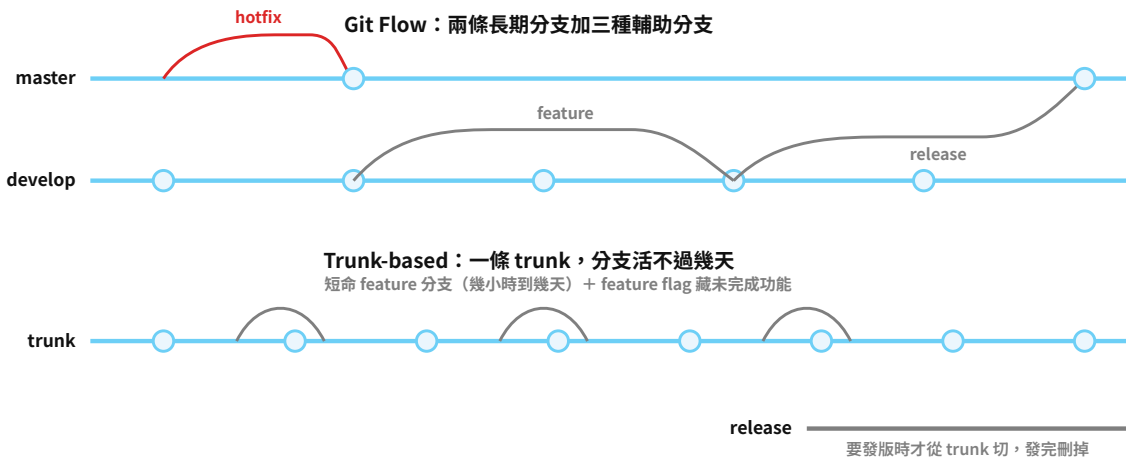
資料庫位址、外部服務的金鑰；每次部署可能不同的一切。改了只要重啟。

分支策略與合併衝突

- 一、 **Git Flow** 與 **Trunk-Based**: 差別是分支活多久
- 二、 三個判準: 發布節奏、團隊規模、測試自動化
- 三、 衝突怎麼解, **merge** 與 **rebase** 差在歷史

兩種模型，差別是分支活多久

上半 Git Flow：master 與 develop 兩條長期分支；下半 Trunk-Based：一條 trunk 加短命分支



上面適合固定版本節奏的產品，下面適合持續交付；差別是分支活多久

Git Flow

feature 從 develop 長出來；release 準備發版，併進 master 與 develop；hotfix 從 master 切出來修線上。

Trunk-Based

短命分支只用來做 code review 與 CI；未完成的功能用 feature flag 藏起來，先合併再打開。

怎麼選？看三個判準

判準	偏 Git Flow	偏 Trunk-Based
發布節奏	固定版本，要同時維護多版	持續交付，只有最新版
團隊規模	大、分工明確、有專職發版	小到中，人人可以推
測試自動化	弱，靠 release 分支手動驗	強， CI 綠燈就能上

你的期末專題是持續交付的 **web** 服務，講師建議 **Trunk-Based** + 短命分支 + **PR**。

“

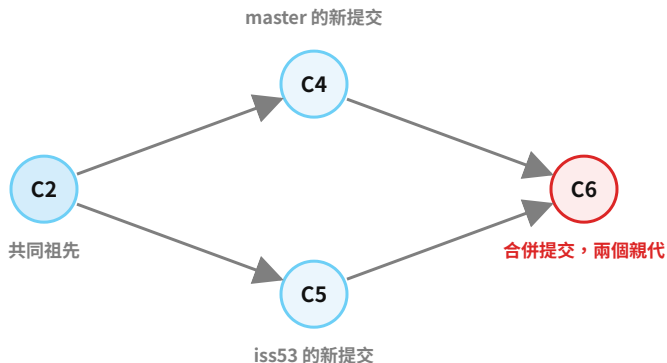
這個模型是 **2010** 年為「有明確版本、要同時支援多個版本在外面跑」的軟體設計的。

做持續交付的 **web app**，建議改用更簡單的流程。

—— Git Flow 作者 2020 年的 Note of reflection (重點摘述) ——

衝突是 **Git** 在問你要哪一版

左邊三個點：共同祖先加兩端，合出一個有兩個親代的合併提交；右邊是衝突時檔案裡的樣子



```
<<<<<< HEAD:index.html
<div id="footer">contact :
email.support@github.com</div>
=====
<div id="footer">
please contact us at
support@github.com
</div>
>>>>>> iss53:index.html
```

三方合併看三個點：兩端加共同祖先；同一檔同一處兩邊都改，Git 就把兩版都留給你，用標記隔開

會衝突

Pro Git 原句：在不同分支中都修改了同一個檔案的同一部分，**Git** 就無法乾淨地合併。

不會衝突

同一個檔案、不同段落，**Git** 自己合得起來。可怕的是積了兩週才合併的衝突面積。

跟著做：解衝突四步

合併噴衝突不要慌，照這四步走一遍

1

看哪些檔案標了 **both modified**

```
$ git status
```

2

打開檔案找標記，決定留上半、留下半，或兩邊合成新的一版

```
<<<<<< HEAD ...你的版本... ===== ...對方的版本... >>>>>> iss53
```

兩邊的業務意圖都要保留，不要隨手挑一邊

3

刪掉三行標記，存檔，把檔案加進索引

```
$ git add path/to/file
```

4

全部解完，完成合併提交

```
$ git commit # rebase 時改成 git rebase --continue
```

rebase 每重放一個提交都可能再解一次；中途放棄用 `git rebase --abort`

結果一樣，歷史不一樣

	merge	rebase
做什麼	三方合併，多一個合併提交	把你的提交搬到對方尾巴重放，之後快進
歷史長相	有分岔有匯合，忠實記錄	一直線，好讀
衝突發生在	合併那一刻，一次解	每重放一個提交都可能要解
不能用的時候	隨時可以	提交已經在你的版本庫之外時，不能

鐵則一條：不要對已經存在於你版本庫之外的提交做 **rebase**。

QUIZ 03

你的 **feature** 分支已經 **push** 給同事一起改。
要更新到最新 **main**，用什麼？

A

git rebase main

B

git merge main

C

git reset --hard main

D

刪掉重開一條

想 30 秒，等一下翻下一頁對答案。

QUIZ 03 · ANSWER

你的 **feature** 分支已經 **push** 給同事一起改。
要更新到最新 **main**，用什麼？

A

git rebase main

B

git merge main

C

git reset --hard main

D

刪掉重開一條

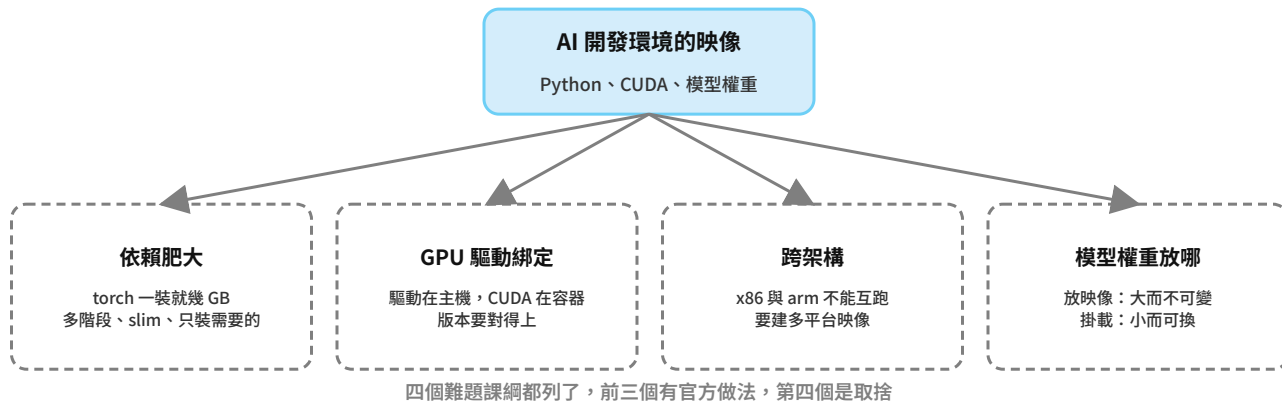
B。這條分支的提交已經在**你的版本庫之外**，**rebase** 會改寫同事手上的歷史，違反鐵則。

AI 環境與稽核實作

- 一、 四個難題：依賴肥大、**GPU** 驅動、跨架構、模型權重
- 二、 權重放哪：三種放法的取捨
- 三、 稽核與瘦身：交出去的是數字，不是感覺

AI 環境卡在四個地方

前三個有官方做法，第四個是取捨；為什麼 AI 映像動不動十幾 GB，看這張就懂



1 CUDA：NVIDIA 的 GPU 運算平台與程式庫。

2 x86 與 arm：兩種 CPU 指令集架構。

3 模型權重：訓練好的參數，一個或多個很大的檔案。

每個難題，對到一個做法

難題	為什麼會發生	做法
依賴肥大	深度學習框架一裝就好幾 GB，加上 CUDA 程式庫	多階段、 slim 基底，只裝需要的變體
GPU 驅動綁定	驅動裝在主機核心層，容器借用它；容器裡的 CUDA 版本要跟主機驅動相容	用 NVIDIA Container Toolkit ，並記錄主機驅動版本
跨架構	容器共用核心， amd64 的映像不能在 arm64 主機直接跑	<code>docker buildx build --platform linux/amd64,linux/arm64</code>
權重放哪	權重幾 GB 到幾十 GB，放進映像就大而不可換	小的放映像，大的掛 Volume 或啟動時下載

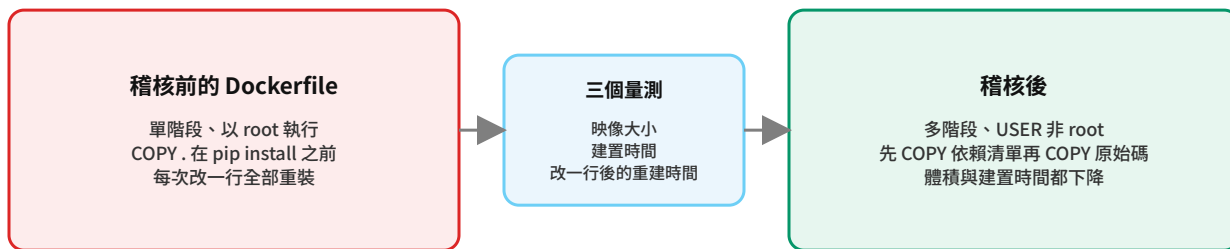
模型權重放哪？三種放法的取舍

放法	好處	代價	適合
烤進映像	一份映像帶著走，不依賴外部	映像巨大、換模型要重建	小模型、離線環境
掛 Volume	映像小、換模型不重建	部署時要先把權重放到那台機器	大模型、常換版
啟動時下載	映像最小	啟動慢、依賴網路與外部服務	雲端、有物件儲存

一句話帶走：AI 映像的難題都是「容器借用主機」這件事的放大版。

實作：稽核一份壞的 Dockerfile

左邊紅框是稽核前的三個問題，右邊綠框是改完的樣子，中間是三個要量的數字



交出去的是前後對照數字，不是感覺；AI 扮 SRE 稽核員抓多餘 Layer、快取失效點與 root

映像大小

一定要下降；常見從 1 GB 以上降到兩三百 MB。

首次建置時間

受下載影響大，只要求記錄與解釋。

改一行後的重建

最誠實的那個數字；從一分鐘以上降到幾秒。

稽核清單七格，每格都要說得出理由

前四格是瘦身，後三格是能不能上線

階段 建置與執行分開，最終映像沒有編譯工具

使用者 有 **USER** 且不是 **root**

資料與埠 程式寫的路徑有規劃 **Volume**，埠有文件化

架構 要上 **x86** 雲端、本機是 **arm** 時有指定 **platform**

順序 依賴清單先於安裝，原始碼最後

基底 **slim** 且釘版本

設定 沒有把 **.env** 或金鑰 **COPY** 進去

AI 用法 要 AI 扮 **SRE** 稽核員，比對它的清單與你的

TAKEAWAY

- 01 容器不是輕的 **VM**，是借用核心的隔離行程。
- 02 映像是交付物，裡面不該有建置工具與 **root**。
- 03 建置變慢多半是順序，不是機器。
- 04 分支活得越久，合併越痛；衝突不是誰的錯。

Reference

- 1 Docker Docs ◦ *Multi-platform builds* ◦ *What is a container?* → 共用核心與跨架構的原句出處
- 2 Docker Docs ◦ *Multi-stage builds* ◦ *Building best practices* → 多階段、釘版本、非 root
- 3 Docker Docs ◦ *Docker build cache* ◦ *Understanding the image layers* → 一層變了後面全重建
- 4 Docker Docs ◦ *Storage* ◦ *Volumes* ◦ *Publishing and exposing ports* → 兩條線與持久化
- 5 Driessen, V. ◦ *A successful Git branching model* (nvie.com) ; *trunkbaseddevelopment.com* → 兩種分支模型與作者反思
- 6 Chacon, S. & Straub, B. ◦ *Pro Git* (繁中版) p.69–72 ◦ p.87–91 → 三方合併、衝突、rebase 鐵則
- 7 The Twelve-Factor App ◦ *III. Config* ; NVIDIA ◦ *Container Toolkit* 文件 → 設定的定義與 GPU 驅動